

Ch01 The Software Crisis, Quality Models, and AI-Era Reliability Engineering

Chapter 01: The Software Crisis, Quality Models, and AI-Era Reliability Engineering

😄 We all know the "law of conservation of matter" in physics; as computer science students, we are even more familiar with the "law of conservation of bugs."

In 2026, writing a piece of code only takes asking AI for 3 seconds; but proving that this code won't bring down the company in the production environment might take 3 months.

1.1 The History of the Software Crisis and Its Reincarnation in the AI Era

Software can benefit humanity, but it can also cause catastrophic disasters. Looking back at history, software defects have triggered severe air crashes, military casualties, and space mission failures costing hundreds of millions of dollars.

Case 1: The Patriot Missile Incident (1991) — Millisecond-level Accumulation of Rounding Error

During the Gulf War in February 1991, an Iraqi Scud missile hit a US barracks in Dhahran, Saudi Arabia, killing **28 soldiers and injuring over a hundred**.

- **Fatal Software Defect:** The Patriot system's internal clock register used a **24-bit floating-point** representation, introducing a tiny truncation error (approx. 0.000000095 seconds) when converting time to tenths of a second.
 - **Disaster Amplification:** The radar system ran continuously for over **100 hours** without a reboot, causing the tiny accuracy error to accumulate to **0.33 seconds**.
 - **Fatal Consequence:** Since the Scud missile travels at Mach 4.2 (1.5 km/s), 0.33 seconds translated to a **600-meter distance deviation**. The radar search window failed to lock onto the target, and no interceptor missile was fired.
 - **SQA Takeaway:** The critical importance of numerical precision, floating-point cumulative errors, and **long-term stress/reliability testing**.
-

Case 2: NASA Mars Climate Orbiter (1998) — The Cost of Units

In 1998, NASA launched the "Mars Climate Orbiter" (costing nearly \$200 million), which lost contact and burned up upon arriving at Mars.

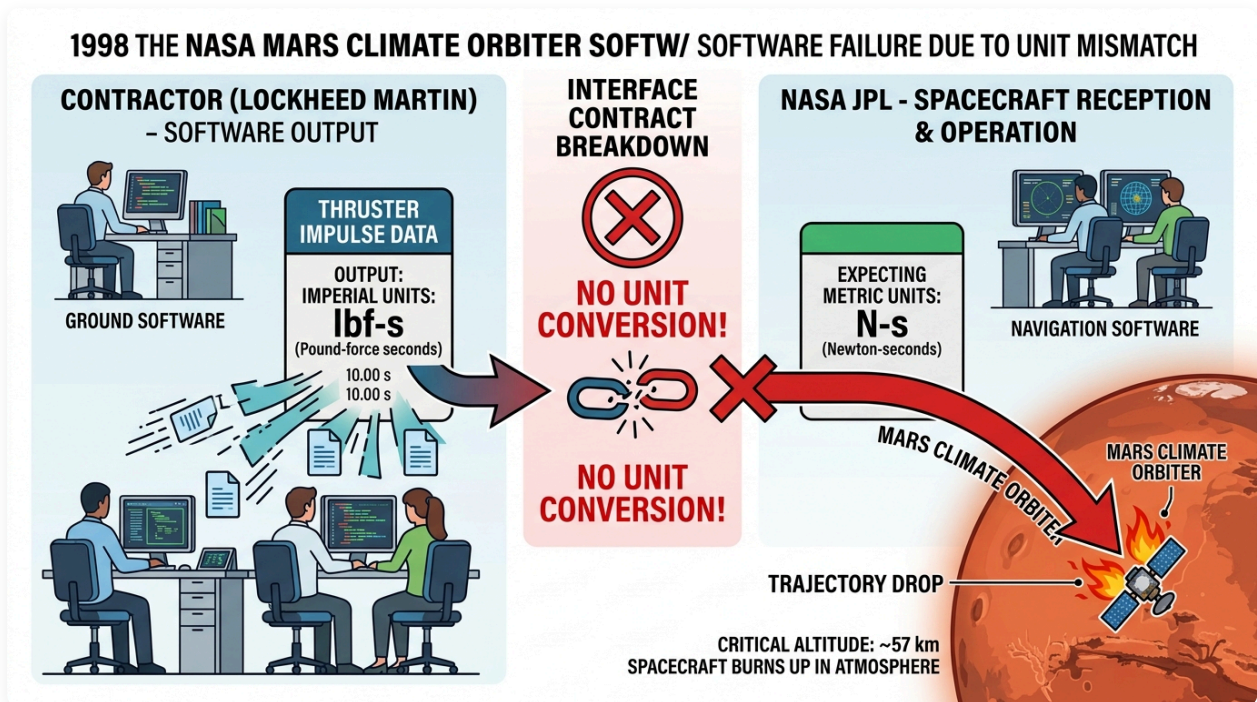


Diagram Explanation: Spacecraft Crash Caused by Broken Cross-Module Interface Contract

- **[Left] Contractor Software (Lockheed Martin):** Ground control software output thruster impulse data in **Imperial units (pound-force seconds, lbf-s)**.
- **[Middle] Interface Contract Breakdown:** Lack of strict interface type definitions and automated unit conversion mechanisms between systems.
- **[Right] NASA JPL Navigation Receiver:** Spacecraft navigation software default-parsed input data in **Metric units (Newton seconds, N-s)**, causing a massive 4.45 times discrepancy in thrust calculation.
- **Disaster Consequence:** The orbit altitude was expected to be 140 km, but plummeted to 57 km, friction-burning and disintegrating in the Martian atmosphere.
- **SQA Takeaway:** The importance of **cross-module interface contracts (Interface Contract)**, strong type checking, and specifications review.

Case 3: China Airlines Nagoya Crash (1994) — Human-Machine Fight for Control

On April 26, 1994, China Airlines Flight CI140 (Airbus A300-622R) crashed while landing at Nagoya Airport, killing **264 people**.

1994 CHINA AIRLINES NAGOYA CRASH: FLIGHT CONTROL SOFTWARE CONFLICT (HMI MODE CONFUSION)

Case Study: Human-Machine Interface Design & System Safety

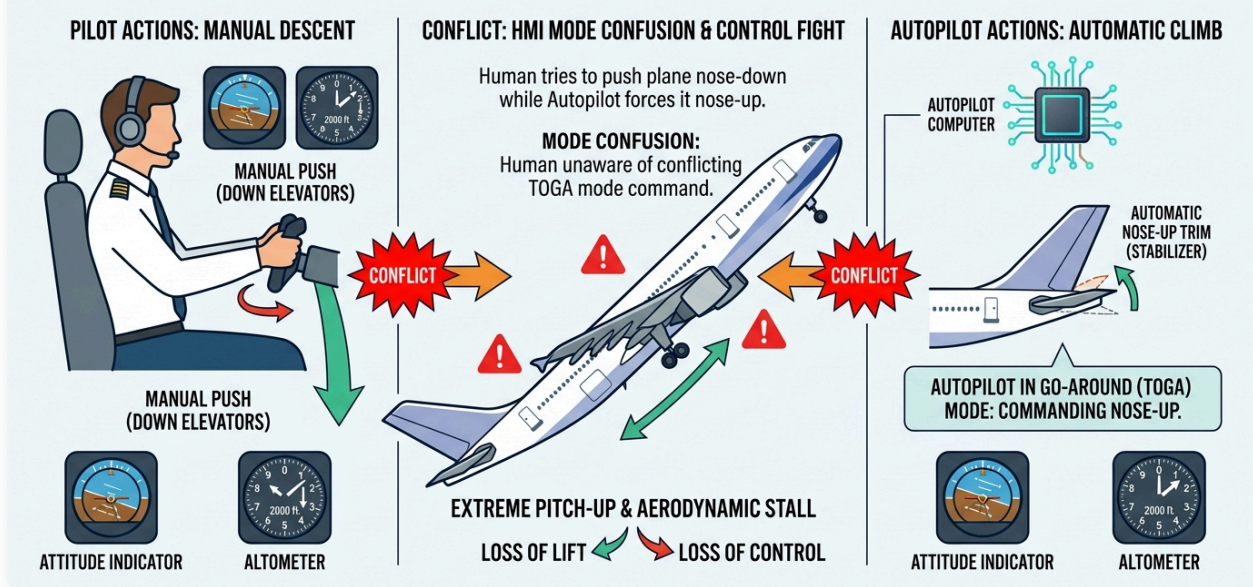


Diagram Explanation: Human-Machine Interface Conflict (HMI Mode Confusion) and Lack of Control Arbitration

- **[Left] Pilot Manual Operation (Manual Push):** After the co-pilot accidentally triggered the "Go-Around (TOGA)" mode, the pilots tried to manually push the control column forward (Down Elevators) to force the nose down for landing.
- **[Right] Flight Control Computer Automatic Trim (Autopilot Automatic Climb):** Because it was in "Go-Around" mode, the onboard flight control computer forced the horizontal stabilizer to trim up to raise the nose for climbing.
- **[Center Conflict] Mode Confusion and Fight for Control (Control Fight):** The pilots did not realize the computer was still executing the Go-Around command, and pilot and machine fought against each other. Eventually, the horizontal stabilizer reached its limit angle, causing a low-altitude **aerodynamic stall** and crash.
- **SQA Takeaway:** Human-computer interaction (HMI/UX) status transparency, feedback on abnormal operations, and automatic control authority arbitration design.

Case 4: Disney's "The Lion King" Game (1994) — A PR Disaster due to Lack of Compatibility Testing

At Christmas in 1994, Disney released "The Lion King" PC game, selling rapidly alongside home computers like Compaq. Tens of thousands of families expected to enjoy it.

- **Fatal Defect:** The game was developed based on a specific video driver (WinG) and was **not adequately tested for compatibility across the diverse hardware environments** prevailing in the market.
- **Disaster Consequence:** A massive number of home PCs crashed to blue screens upon startup. Angry parents flooded customer support on Christmas Day, severely damaging

the brand's reputation.

- **SQA Takeaway:** The importance of environment diversity validation and **compatibility testing (Compatibility Testing)**, which prompted Microsoft to later develop the standardized DirectX architecture.

Case 5: AI-Written Code — Why Can It Bankrupt a Company in 3 Seconds?

Before diving into theory, let's look at a "bank e-wallet transfer service" generated by modern top-tier AI (GPT-4 / Claude 3.5):

```
public class WalletService {
    private double balance = 1000.0; // Initial account balance $1000

    // AI-generated transfer method: contains basic checks and deduction logic
    public boolean transfer(double amount) {
        if (amount <= 0) {
            return false;
        }
        if (balance >= amount) {
            // Simulate network delay or database query
            try { Thread.sleep(10); } catch (InterruptedException e) {}

            balance -= amount;
            return true;
        }
        return false;
    }

    public double getBalance() {
        return balance;
    }
}
```

Live Attack Experiment

This code looks neatly formatted and logical. Typical unit tests like `assert transfer(100) == true` pass with green lights.

However, when we send "transfer \$100" requests concurrently using 50 threads:


```
// 50 users concurrently transferring $100 (Total withdrawal demand $5000, but
ExecutorService executor = Executors.newFixedThreadPool(50);
for (int i = 0; i < 50; i++) {
    executor.submit(() -> walletService.transfer(100.0));
}
executor.shutdown();
executor.awaitTermination(5, TimeUnit.SECONDS);

System.out.println("Final Balance: " + walletService.getBalance());
```

Execution Result:

⚠️ **Final Balance: -3200.0** (The account originally had only 1000, *but* 4200 was withdrawn, causing a severe overdraft!)

💣 Shocking Reflections:

1. **AI will not proactively consider concurrency safety and state invariants:** AI only generated sequential code based on common code snippets, lacking atomic protection for thread-safety.
2. **AI-generated tests are often "self-fulfilling false green lights":** If you ask AI to write tests for this code, it will only write single-threaded tests, passing with 100% coverage, giving you a false sense of security before putting a time bomb online.
3. **The true mission of software engineers in 2026:**
 - Writing code (Coding) is no longer a scarce skill;
 - "Defining specifications and invariants," "designing destructive tests," and "building automated quality guardrails" are the irreplaceable core values of human engineers!

1.1.5 The Essence of the Software Crisis: From 1968 to 2026

- In 1968, the NATO conference first proposed the "Software Crisis": hardware was advancing rapidly, while software development faced **budget overruns, delayed schedules, low quality, and maintenance difficulties**.
- **The New Software Crisis in 2026 (The Verification Bottleneck):**
 - AI has accelerated code writing by 10x, causing a geometric explosion in code output.
 - However, human capacity to verify code, review specifications, and ensure system reliability has not automatically increased by 10x!
 - **The Software Crisis has not disappeared; it has simply evolved into a "Reliability Crisis".**

Data and Evidence: AI speeds up code creation, but has quality improved?

This is a critical engineering mindset question. When development teams heavily rely on AI coding assistants, do we really get higher quality software? **The answer might be quite the opposite.** Several authoritative empirical studies and academic surveys in recent years provide shocking data:

1. Deterioration of Code Maintainability: GitClear Longitudinal Study (2020–2026)

- GitClear analyzed **150 million lines of code** in Git commits and found that since AI assistants became popular:
 - **Code duplication** has risen exponentially.
 - **"Moved Lines" (a key indicator of refactoring) has dropped significantly**, meaning engineers are less active in refactoring and cleaning up legacy code.
 - **Code churn** (newly written code deleted or rewritten shortly after) has increased dramatically. This proves AI generates code that seems working but is actually fragile, bringing heavy **long-term maintainability debt**.

2. 52% Error Rate and "False Sense of Security": Purdue University Empirical Study

- A Purdue University research team evaluated ChatGPT's performance on 517 Stack Overflow software engineering questions:
 - The results showed that **52% of AI answers contained incorrect code or information**.
 - More alarmingly, because the AI's tone was extremely polite, structured, and "highly logical," up to **39.3% of users still preferred and accepted the AI's incorrect answers**.
 - This gives engineers a false sense of security, leaking bugs and vulnerabilities directly into production systems without rigorous verification.

3. 40% Security Vulnerability Risk: New York University (NYU) Academic Research

- Researchers performed automated security scans on AI-generated code (based on the Common Weakness Enumeration, CWE standard):
 - The study found that when generated without specific security prompts, **approximately 40% of AI-generated code contained known security vulnerabilities** (such as buffer overflow, SQL injection, concurrency race conditions).

Conclusion:

In the 2026 AI era, the bottleneck of software development has shifted entirely from "whether code can be written (Writing)" to "whether the code is correct (Verification)". If developers lack quality assurance knowledge and blindly trust AI's "green lights," software systems will collapse rapidly.

 **Software and cathedrals are much the same – first we build them, then we pray.**
(Sam Redwine)

Concept Check Question (CCQ 1)

Question

What was the fundamental software cause of the Patriot Missile system's intercept failure at the Dhahran base in 1991?

- A) Communication network outage preventing the radar from sending commands to the missile launcher.
- B) Floating-point rounding error in the 24-bit clock register accumulating to 0.33 seconds after 100 hours of continuous operation.
- C) Memory leak in the code causing the operating system to crash.
- D) Radar algorithm misidentifying a US fighter jet as an enemy Scud missile.

► [Click to view Concept Check Question Answer and Explanation](#)

1.2 The Nature of Software and Quality Dimensions (Software Four Elements & Garvin's Five Quality Views)

1.2.1 The Four Elements of Software (IEEE 610.12)

What exactly is software? Is it merely executable binaries or source code? According to the authoritative definition of IEEE (Standard 610.12), software is a complete, systematic engineering product:

Software: Computer **programs**, **procedures**, and possibly associated **documentation** and **data** pertaining to the operation of a computer system.

EDUCATIONAL SOFTWARE ENGINEERING INFOGRAPHIC: 4 CORE ELEMENTS OF SOFTWARE (IEEE STANDARD)

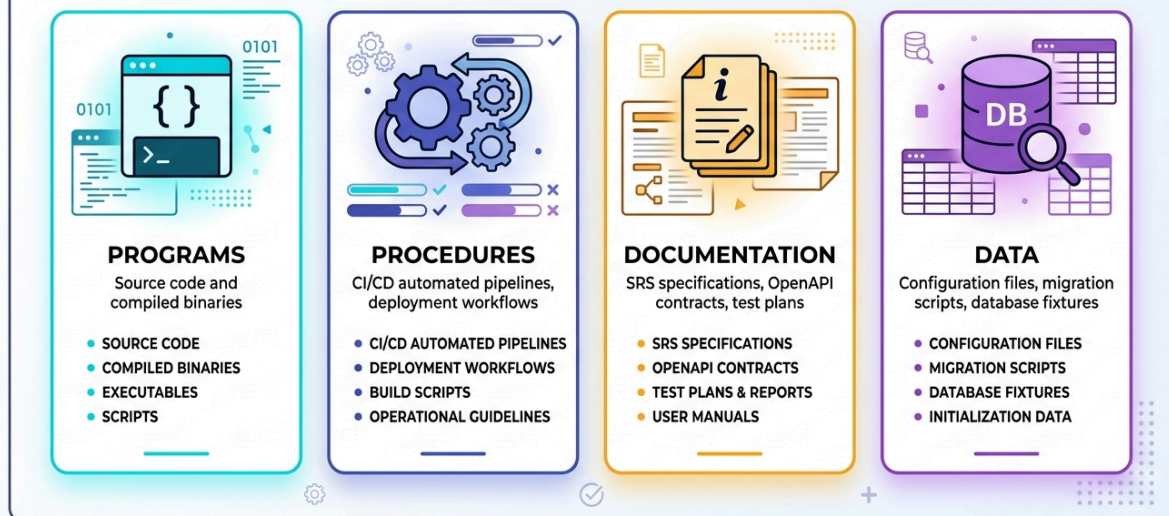


Diagram Explanation: Four Core Elements of Software (IEEE 610.12)

1. **Programs (Code):** Includes source code, compilation outputs (Bytecode/Binary), and execution scripts, which implement business logic and algorithms.
2. **Procedures (Operations):** Includes CI/CD automation build scripts, deployment procedures, runbooks, and release standards.
3. **Documentation (Specs):** Includes Software Requirement Specifications (SRS), OpenAPI interface contracts, architecture design diagrams, and test plans (specifications as living documents).
4. **Data (Config & Fixtures):** Includes database migration schemas, environment variable configuration files, and test datasets (Test Fixtures).

1.2.2 What is Quality? David Garvin's Five Quality Views

When discussing "software quality," Harvard Business School professor David Garvin pointed out in *Managing Quality* that quality is not a single dimension, but a multi-dimensional concept woven from different perspectives:

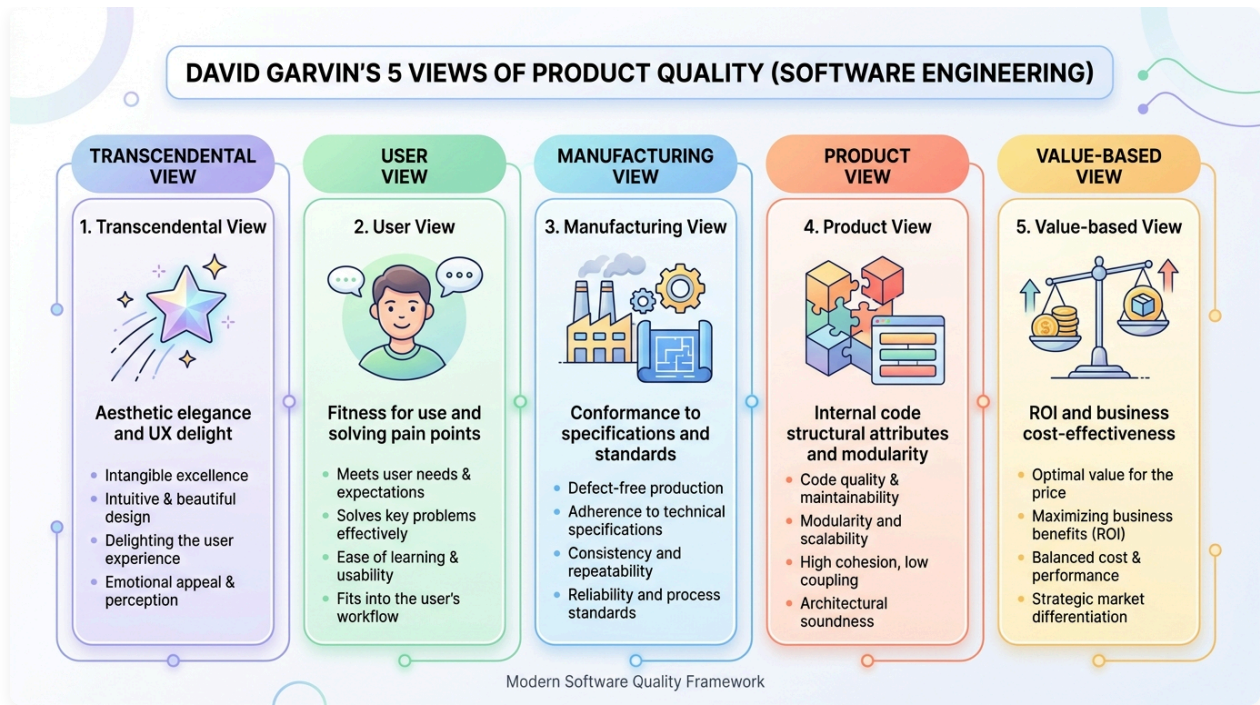


Diagram Explanation: David Garvin's Five Quality Views

- 1. Transcendental View:** Cannot be precisely quantified, but is recognized immediately through experience as refined, elegant, and intuitive (e.g., smooth UI/UX with micro-interactions).
- 2. User View (Fitness for Use):** Whether the software addresses actual user pain points, fits business requirements, and delivers tangible benefits.
- 3. Manufacturing View (Conformance):** Whether software products and engineering processes conform 100% to specifications, static checks, and Quality Gates.
- 4. Product View (Architecture):** Internal structural characteristics of the product, such as high cohesion, low coupling, strong typing, testability, and maintainability.
- 5. Value-based View (ROI):** Whether the commercial value and output of the software significantly exceed the total cost of development, testing, and operations.

Quality View	Core Definition	Software Engineering Example	Consequence of Ignoring This View
Transcendental View	Cannot be quantified; recognized immediately through elegant experience	Smooth UI/UX, refined micro-animations	Software feels poorly made, cold, and hard to use
User View	Meets actual user needs and expectations (Fitness for Use)	Solves user pain points; intuitive workflow	Highly functional but no one wants to use it (Shelfware)
Manufacturing View	Conforms to engineering specs and standard processes (Conformance)	Clean Code compliance, zero specification drift, passing Quality Gates	When specs have loopholes, a perfect piece of garbage is built
Product View	Internal technical characteristics and architectural build quality	High cohesion, low coupling, strong typing, low cyclomatic complexity	Architectural decay; modifying a small feature triggers a full collapse
Value-based View	Cost-to-value ratio the customer is willing to pay (ROI)	Software business value exceeds development and operation costs	Development costs spiral out of control; commercially unviable

👍 Programs must be written for people to read, and only incidentally for machines to execute. — *Abelson & Sussman*

👍 Quality is not an act, it is a habit. — *Aristotle*

Concept Check Question (CCQ 2)

Question

An e-commerce app developed by a project team perfectly meets every single requirement in the contract (conforms to the Manufacturing View). However, because the underlying architecture is highly coupled and lacks any unit tests, the team finds they must rewrite the entire system six months later just to add a new promotion feature. Which quality view from Garvin does this software severely fail?

- A) Product View
- B) Manufacturing View
- C) Legal Contract View
- D) Transcendental View

► Click to view Concept Check Question Answer and Explanation

1.3 Core Concepts of Software Quality Engineering: V&V, Cost of Quality (CoQ), and Shift-Left Testing

1.3.1 Verification vs. Validation (V&V)

The ultimate question of software testing and quality assurance:

-  **Verification:** *Are we building the product **right**?* (Are we building the software correctly?)
-  **Validation:** *Are we building the **right** product?* (Are we building the correct software?)

- **Verification:** Ensuring the software output matches the specifications set in the previous stage (e.g., checking if code matches design documents, or if unit tests match component specs).
 - **Validation:** Ensuring the software meets the user's actual business needs (e.g., acceptance testing, usability testing, beta testing on-site).
-

1.3.2 Software Cost of Quality (CoQ) and the 1:10:100 Law

Software quality is not "the more perfect, the better," but a balance between costs and benefits. In software quality management, the Cost of Quality (CoQ) is divided into **Conformance Costs** and **Non-Conformance Costs**:

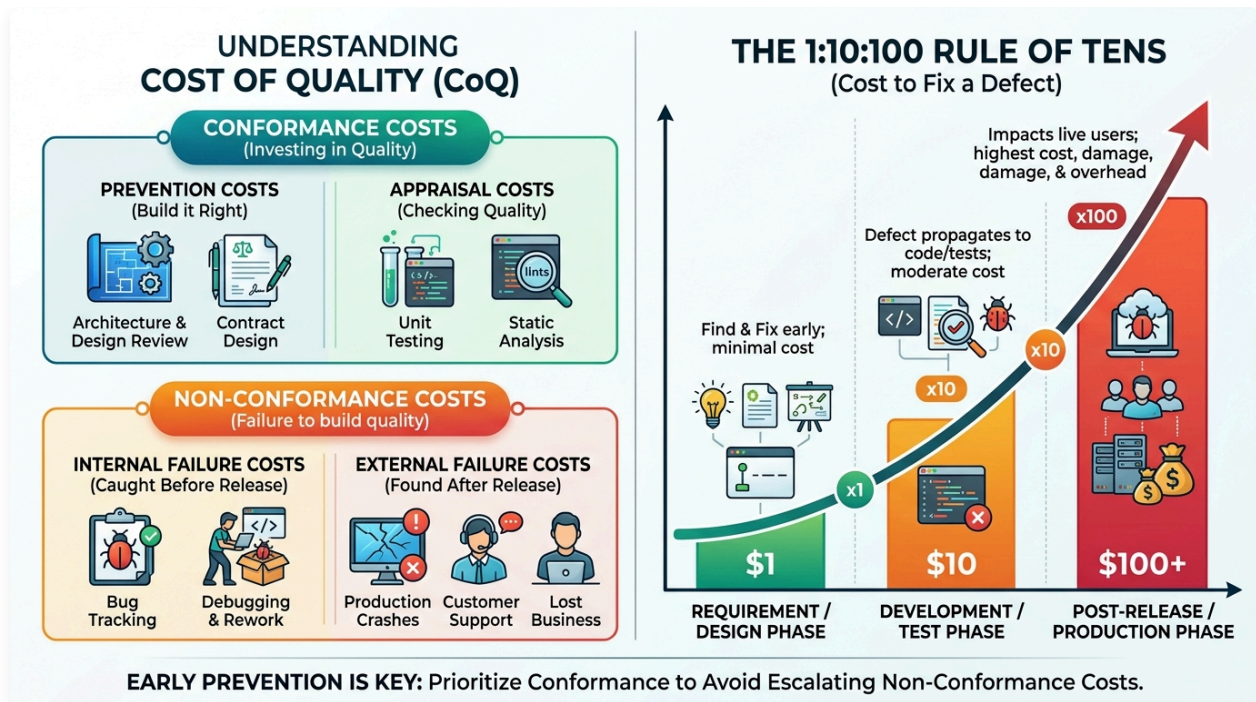


Diagram Explanation: Cost of Quality (CoQ) Framework and the 1:10:100 Rule of Defect Multiplier

- **Conformance Costs (Active Investment in Quality):**
 - **Prevention Costs:** Architecture reviews, Design by Contract, developer training, and static analysis guidelines.
 - **Appraisal Costs:** Unit tests, static code analysis (SonarQube), and peer code reviews.
- **Non-Conformance Costs (Painful Price of Ignoring Quality):**
 - **Internal Failure Costs:** Bug fixes (debugging), refactoring, and retesting before release.
 - **External Failure Costs:** Production system downtime (outage), customer compensation claims, emergency hotfixes, and brand reputation bankruptcy.
- **1:10:100 Law (The Rule of Tens):**
 - Finding and fixing a defect during the **Requirements/Design** phase costs \$1.
 - Delaying it to the **Development/Testing** phase increases the cost to \$10.
 - If it leaks into the **Production** phase, the fix cost and disaster damage will skyrocket to 100to1000+!
- **Shift-Left Testing:** Performing quality activities as early as possible in the lifecycle is the most effective way to reduce the total cost of ownership.

1.4 Quality Gates in the Software Development Life Cycle (SDLC & CI/CD Quality Governance)

The core philosophy of software engineering is: "Quality is built-in, not tested-in."

1.4.1 Traditional Models and the V-Model: Symmetry and Early Test Planning

In the traditional linear model (Waterfall), testing was often delayed until coding was finished, falling into the expensive 1:10:100 trap. To address this, the **V-Model** establishes strict symmetry and parallel planning between development stages and testing levels:

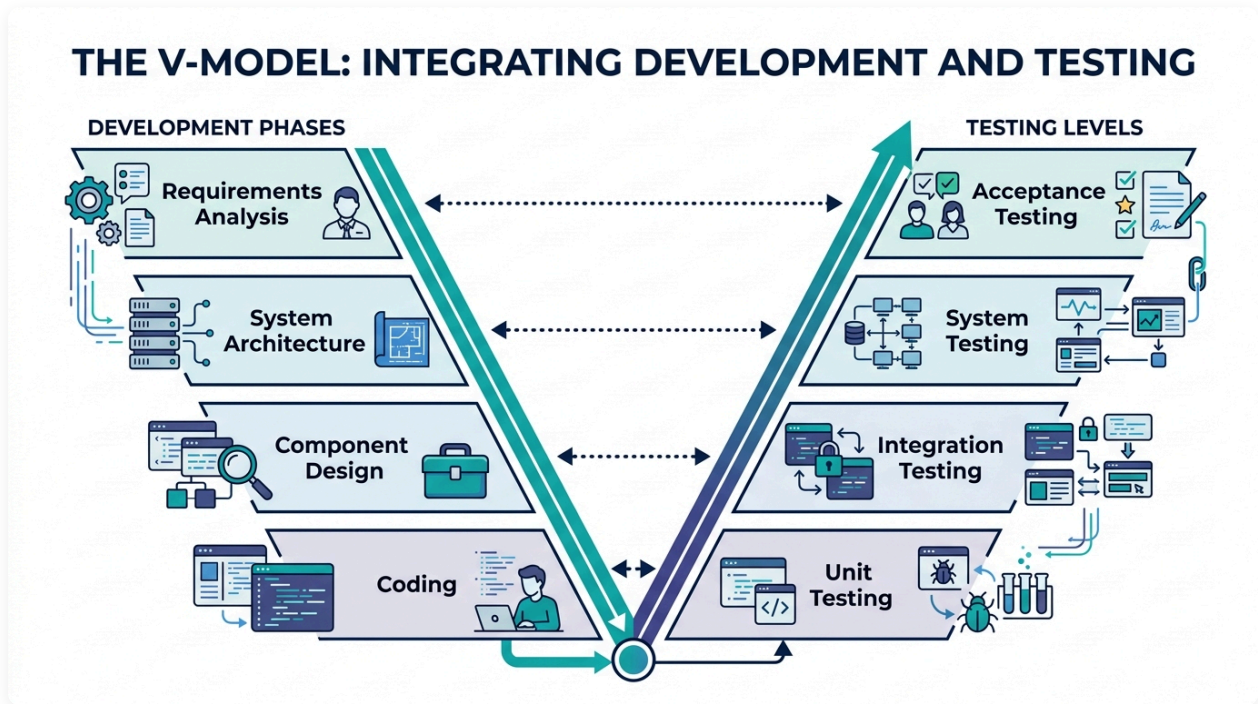


Diagram Explanation: Symmetry of Development and Testing Levels in the V-Model

- **Left: Development Phases (Verification) → Right: Testing Levels (Validation)** are symmetrically mapped:
 - i. **Requirements Analysis** → Plan and design **Acceptance Testing** concurrently.
 - ii. **System Architecture** → Plan and design **System Testing** concurrently.
 - iii. **Component Design** → Plan and design **Integration Testing** concurrently.
 - iv. **Coding** → Implement and run **Unit Testing**.
- **Core Value:** Before writing the first line of business code, the specification and boundaries of acceptance and integration tests are already established alongside the architecture diagrams.

1.4.2 Modern Agile and DevOps CI/CD Continuous Quality Gates

In modern cloud-native and microservice eras, software is delivered daily or even hourly. Quality assurance has evolved into automated **Continuous Quality Gates** in the CI/CD pipeline:

CONTINUOUS QUALITY GATES IN MODERN DEVOPS CI/CD PIPELINE

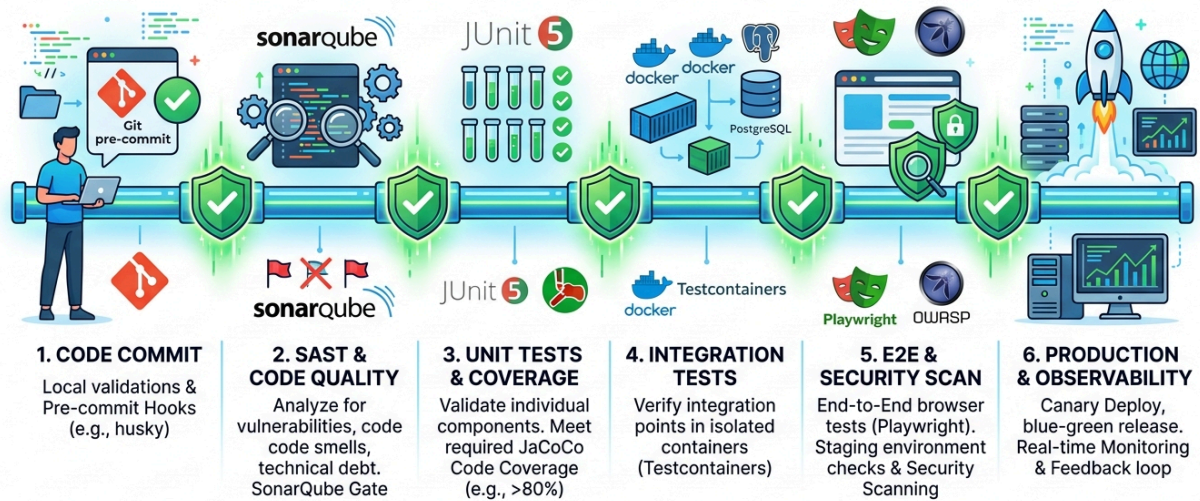


Diagram Explanation: 6 Continuous Quality Gates in Modern DevOps CI/CD Pipeline

- 1. Code Commit Gate:** Local Git pre-commit hooks automatically execute code formatting and quick static syntax checks.
- 2. SAST Static Code Quality Gate:** SonarQube / SpotBugs scans for code smells, technical debt, and OWASP security vulnerabilities.
- 3. Unit Tests & Coverage Gate:** JUnit 5 executes millisecond-level unit tests, and JaCoCo verifies line and branch coverage thresholds (e.g., > 80%).
- 4. Integration Tests Container Gate:** Testcontainers launches real Docker containers (e.g., PostgreSQL / Redis) to verify database access and API contracts.
- 5. E2E & Security Scan Acceptance Gate:** Playwright automatically simulates user flows, paired with OWASP ZAP for dynamic penetration scanning.
- 6. Production & Observability Self-Healing Gate:** Smooth releases via canary or blue-green deployments, with real-time P99 latency and exception alerts monitored by observability systems.

1.5 Modern Software Quality Models (ISO 9126 → ISO 25010)

Every industry has its unique quality model. For instance, a manufacturer of cheap plastic chairs will not prioritize "repairability" as a core quality metric; chairs are replaced rather than repaired. However, the automotive industry must place "safety" and "maintainability" at the absolute top.

DIFFERENT INDUSTRIES & PRODUCTS HAVE UNIQUE QUALITY MODELS



Diagram Explanation: Comparison of Quality Models Across Industries

1. **Automobile:** Prioritizes **Safety**, **Maintainability**, and **Repairability**.
2. **Luxury Mechanical Watch:** Prioritizes **Precision**, **Craftsmanship**, and **Transcendental Elegance**.
3. **Fast Food Restaurant:** Prioritizes **Speed**, **Consistency**, and **Value**.
4. **Software Systems:** Prioritizes **Scalability**, **Security**, **Portability**, and **Fault Tolerance**.

1.5.1 ISO 25010 Eight Product Quality Characteristics

The International Organization for Standardization originally established **ISO 9126** (defining 6 characteristics). The modern **ISO 25010 (SQuaRE standard)** expands this into **8 Product Quality Characteristics**:

ISO 25010 PRODUCT QUALITY CHARACTERISTICS

(Focusing only the first level)



Diagram Explanation: ISO 25010 Eight Product Quality Characteristics (Core Dimensions)

1. Functional Suitability

Degree to which a product provides functions that meet stated and implied needs when used under specified conditions:

- **Functional Completeness:** The degree to which the set of functions covers all specified tasks and user objectives.
- **Functional Correctness:** The degree to which a product provides the correct results with the needed degree of precision (e.g., an ATM withdrawal not only dispenses cash, but the account deduction must match the exact amount).
- **Functional Appropriateness:** The degree to which the functions facilitate the accomplishment of specified tasks and objectives (e.g., forcing online chat and live streaming into a markdown editor violates functional appropriateness).

2. Reliability

Degree to which a system performs specified functions under specified conditions for a specified period of time:

- **Maturity:** Degree to which a system meets needs for reliability under normal operation (low failure rate).
- **Fault Tolerance:** Degree to which a system operates as intended despite the presence of hardware or software faults (e.g., throwing a user-friendly error instead of crashing when receiving deformed JSON).
- **Recoverability:** Degree to which, in the event of an interruption or a failure, a product can recover the data directly affected and re-establish the desired state of the system (e.g.,

recovering data consistency in seconds via WAL logs after a database crash).

3. Performance Efficiency

Performance relative to the amount of resources used under stated conditions:

- **Time Behavior:** Response and processing times and throughput rates of a product when performing its functions (e.g., API P99 response time < 200ms).
- **Resource Utilization:** The amounts and types of resources used by a product when performing its functions (CPU, memory, disk I/O, network bandwidth).
- **Capacity:** Maximum limits of a product parameter (e.g., maximum concurrent connections or database storage limits).

4. Usability

Degree to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction:

- **Appropriateness Recognizability:** Degree to which users can recognize whether a product is appropriate for their needs.
- **Learnability:** Degree to which a product can be used by specified users to achieve specified goals of learning to use the product.
- **Operability:** Degree to which a product has attributes that make it easy to operate and control.
- **User Error Protection:** Degree to which a system protects users against making errors (e.g., a confirmation prompt before formatting a disk).

5. Security

Degree to which a product protects information and data so that persons or other products have the degree of data access appropriate to their types and levels of authorization:

- **Confidentiality:** Degree to which a product ensures that data are accessible only to those authorized to have access (e.g., salted password hashing, HTTPS encryption).
- **Integrity:** Degree to which a system prevents unauthorized access to, or modification of, computer programs or data.
- **Non-repudiation:** Degree to which actions or events can be proven to have taken place, so that the events or actions cannot be repudiated later (e.g., digital signatures, immutable audit logs).
- **Authenticity and Accountability.**

6. Maintainability

Degree of effectiveness and efficiency with which a product can be modified by the intended maintainers:

- **Modularity:** Degree to which a system is composed of discrete components such that a change to one component has minimal impact on other components.
- **Analyzability:** Degree of ease with which the impact of a planned change can be assessed, or defects diagnosed (e.g., good structured logging and distributed tracing).
- **Modifiability:** Degree to which a product can be effectively and efficiently modified without introducing defects or degrading existing product quality.
- **Testability:** Degree of ease with which test criteria can be established and tests performed to determine whether those criteria have been met (high cohesion and low coupling architectures yield high testability).

7. Portability

Degree of ease with which a product can be effectively and efficiently transferred from one hardware, software, or other operational environment to another:

- **Adaptability:** Degree to which a product can effectively and efficiently be adapted for different or evolving operational environments without additional actions.
- **Installability:** Degree of effectiveness and efficiency with which a product can be successfully installed and/or uninstalled in a specified environment.
- **Replaceability:** Degree to which a product can replace another specified software product for the same purpose in the same environment (e.g., using Docker and Testcontainers to achieve 100% consistency across developer machines, CI servers, and production).

8. Compatibility

Degree to which a product can exchange information with other products, and/or perform its required functions while sharing the same hardware or software environment:

- **Co-existence:** Degree to which a product can perform its required functions efficiently while sharing a common environment and resources with other products, without detrimental impact on any other product.
- **Interoperability:** Degree to which two or more systems can exchange information and mutually use the information that has been exchanged (via REST, GraphQL, LDAP, etc.).

1.5.2 ISO 25010 Quality Map & 16-Week Course Testing Technology Map

Software testing is never about blindly hitting code. Each testing and engineering technique taught in this course builds an automated defense line for specific dimensions of the quality model:

ISO 25010 Quality Characteristic	Core Sub-characteristics	Course Testing & Engineering Technology
Functional Suitability	Completeness, Correctness, Appropriateness	Equivalence Partitioning (EP), Boundary Value Analysis (BVA), JUnit 5, BDD (Cucumber)
Reliability	Maturity, Fault Tolerance, Recoverability	Assertions, jqwik Property-Based Testing , Chaos Engineering
Maintainability	Modularity, Analyzability, Modifiability, Testability	Static analysis (SonarQube/SpotBugs), Mutation Testing (PITest) , Dependency Decoupling
Security	Confidentiality, Integrity, Non-repudiation, Authenticity	Static Application Security Testing (SAST), Fuzzing (Jazzer)
Performance Efficiency	Time behavior, Resource utilization	k6 / Apache JMeter high-concurrency stress testing , GC monitoring, memory leak analysis
Compatibility	Co-existence, Interoperability	Microservices Contract Testing (Pact) , Cross-version compatibility
Portability	Adaptability, Installability, Replaceability	Testcontainers containerized testing , Cloud-native multi-environment verification
Usability	Appropriateness, recognizability, Learnability, Operability	Playwright E2E testing , User workflow automation

1.6 Comprehensive Practice and Brainstorming

1. Reflections on Quality in the AI Era:

- When generative AI can produce code with complete Javadoc in seconds, why does the value of software testing engineers increase significantly? Please explain from the aspects of the "Test Oracle Problem" and "Self-Fulfilling Confirmation Bias".

2. ISO 25010 Dimensional Analysis:

- Analyze the following scenario: "After a backend database crash and reboot, a microservice system automatically reconnects within 5 seconds and successfully retries failed messages without losing any transactions." Which quality characteristics in ISO 25010 does this embody?

3. Patriot Missile and Numerical Precision:

- Write a small Java code snippet to continuously add `0.1` a total of 1,000,000 times, and compare the result with `100000.0`. Observe the accumulation of floating-point errors over long-term execution.